

## 1. Data Ingestion & Quality Profiling

### 1.1 Load & Inspect

1. Read car\_prices.csv into a pandas DataFrame.

```
1 import pandas as pd
2 df = pd.read_csv('/content/car_prices.csv')
```

```
1 # Display first 5 rows
2 print("First 5 Rows:")
3 display(df.head())
```

First 5 Rows:

|   | year | make  | model               | trim       | body  | transmission | vin               | state | condition | odometer | color | interior | seller                                |
|---|------|-------|---------------------|------------|-------|--------------|-------------------|-------|-----------|----------|-------|----------|---------------------------------------|
| 0 | 2015 | Kia   | Sorento             | LX         | SUV   | automatic    | 5xyktca69fg566472 | ca    | 5.0       | 16639.0  | white | black    | kia motor america                     |
| 1 | 2015 | Kia   | Sorento             | LX         | SUV   | automatic    | 5xyktca69fg561319 | ca    | 5.0       | 9393.0   | white | beige    | kia motor america                     |
| 2 | 2014 | BMW   | 3 Series            | 328i SULEV | Sedan | automatic    | wba3c1c51ek116351 | ca    | 45.0      | 1331.0   | gray  | black    | financial service remarketing (lease) |
| 3 | 2015 | Volvo | S60                 | T5         | Sedan | automatic    | yv1612tb4f1310987 | ca    | 41.0      | 14282.0  | white | black    | volvo rep/wholesaler                  |
| 4 | 2014 | BMW   | 6 Series Gran Coupe | 650i       | Sedan | automatic    | wba6b2c57ed129731 | ca    | 43.0      | 2641.0   | gray  | black    | financial service remarketing (lease) |

```
1 # Display data types
2 print(df.dtypes)
```

```
year          int64
make          object
model         object
trim          object
body          object
transmission  object
vin           object
state         object
condition     float64
odometer      float64
color         object
interior      object
seller        object
mmr           float64
sellingprice  float64
saledate      object
dtype: object
```

```
1 # Display record count
2 print('Total Records:', len(df))
```

Total Records: 558837

## 1.2 Understanding the Data Structure

```
1 # Check the shape of the dataset (rows and columns).
2 print('Dataset Shape:', df.shape)
```

Dataset Shape: (558837, 16)

```
1 # Display column names and data types.
2 print('Columns:', df.columns)
```

Columns: Index(['year', 'make', 'model', 'trim', 'body', 'transmission', 'vin', 'state', 'condition', 'odometer', 'color', 'interior', 'seller', 'mmr', 'sellingprice', 'saledate'], dtype='object')

```
1 # Column datatypes and record count.
2 print(df.info())
```

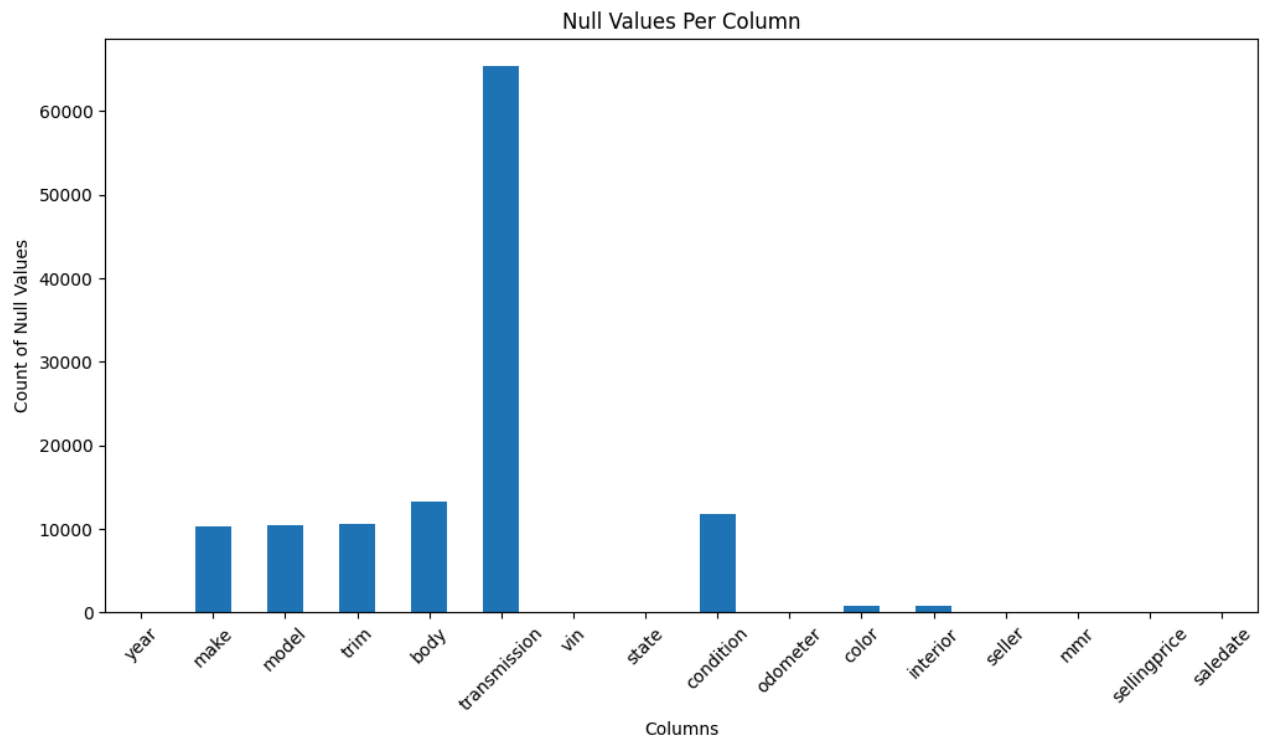
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 558837 entries, 0 to 558836
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   year            558837 non-null  int64
1   make            548536 non-null  object
2   model           548438 non-null  object
3   trim            548186 non-null  object
4   body            545642 non-null  object
5   transmission    493485 non-null  object
6   vin             558833 non-null  object
7   state           558837 non-null  object
8   condition       547017 non-null  float64
9   odometer        558743 non-null  float64
10  color           558088 non-null  object
11  interior        558088 non-null  object
12  seller          558837 non-null  object
13  mmr             558799 non-null  float64
14  sellingprice    558825 non-null  float64
15  saledate        558825 non-null  object
dtypes: float64(4), int64(1), object(11)
memory usage: 68.2+ MB
None
```

## 1.3 Missing & Anomaly Detection

```
1 # Quantify nulls per column; visualize with a bar chart or heatmap.
2 null_values = df.isnull().sum()
3 print(null_values)
```

```
year            0
make           10301
model          10399
trim           10651
body           13195
transmission    65352
vin             4
state           0
condition      11820
odometer        94
color           749
interior        749
seller          0
mmr            38
sellingprice    12
saledate        12
dtype: int64
```

```
1 import matplotlib.pyplot as plt
2 plt.figure(figsize=(12,6))
3 null_values.plot(kind='bar')
4 plt.title('Null Values Per Column')
5 plt.xlabel('Columns')
6 plt.ylabel('Count of Null Values')
7 plt.xticks(rotation=45)
8 plt.show()
```



```

1 # Quantify nulls per column
2 null_counts = df.isnull().sum()
3 null_percentages = (df.isnull().sum() / len(df)) * 100
4
5 null_info = pd.DataFrame({
6     'Null Count': null_counts,
7     'Null Percentage': null_percentages
8 }).sort_values(by='Null Count', ascending=False)
9
10 print("Null values per column:")
11 display(null_info[null_info['Null Count'] > 0])

```

Null values per column:

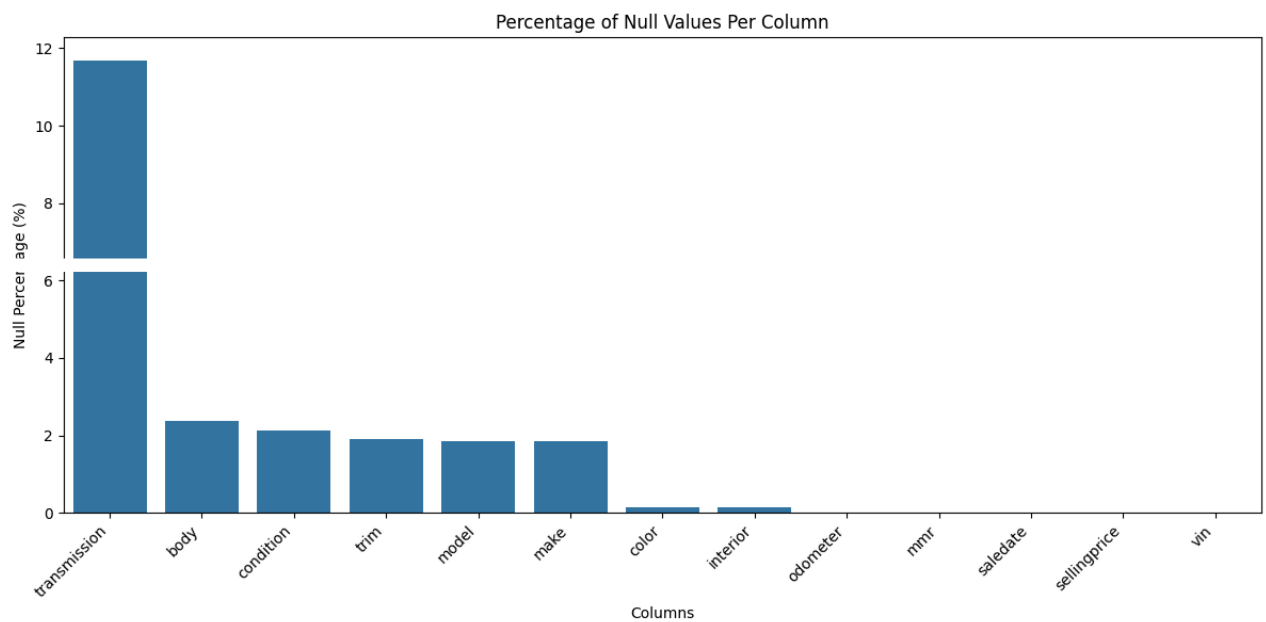
|                     | Null Count | Null Percentage |
|---------------------|------------|-----------------|
| <b>transmission</b> | 65352      | 11.694287       |
| <b>body</b>         | 13195      | 2.361154        |
| <b>condition</b>    | 11820      | 2.115107        |
| <b>trim</b>         | 10651      | 1.905922        |
| <b>model</b>        | 10399      | 1.860829        |
| <b>make</b>         | 10301      | 1.843292        |
| <b>color</b>        | 749        | 0.134028        |
| <b>interior</b>     | 749        | 0.134028        |
| <b>odometer</b>     | 94         | 0.016821        |
| <b>mmr</b>          | 38         | 0.006800        |
| <b>saledate</b>     | 12         | 0.002147        |
| <b>sellingprice</b> | 12         | 0.002147        |
| <b>vin</b>          | 4          | 0.000716        |

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Visualize nulls with a bar chart
5 plt.figure(figsize=(12, 6))
6 sns.barplot(x=null_info[null_info['Null Count'] > 0].index, y=null_info[null_info['Null Count'] > 0]['Null Percentage'])
7 plt.title('Percentage of Null Values Per Column')
8 plt.xlabel('Columns')
9 plt.ylabel('Null Percentage (%)')
10 plt.xticks(rotation=45, ha='right')

```

```
11 plt.tight_layout()
12 plt.show()
```



## ✓ Resolving Null Values

Based on the analysis of null percentages and data types, the following strategies will be applied:

- **Drop rows:** For columns `vin`, `sellingprice`, and `saledate`, which have a very small number of missing values and are critical identifiers or target variables, rows containing these nulls will be dropped.
- **Impute with mode:** For categorical columns (`transmission`, `body`, `trim`, `model`, `make`, `color`, `interior`) with missing values, the mode (most frequent value) will be used for imputation.
- **Impute with median:** For numerical columns (`condition`, `odometer`, `mmr`) with missing values, the median will be used for imputation to be robust against outliers.

```
1 # Drop rows with nulls in vin, sellingprice, saledate
2 initial_rows = df.shape[0]
3 df.dropna(subset=['vin', 'sellingprice', 'saledate'], inplace=True)
4 print(f"Dropped {initial_rows - df.shape[0]} rows with nulls in 'vin', 'sellingprice', or 'saledate'.")
5
6 # Impute categorical columns with mode
7 categorical_cols_to_impute = ['transmission', 'body', 'trim', 'model', 'make', 'color', 'interior']
8 for col in categorical_cols_to_impute:
9     if df[col].isnull().any():
10         mode_value = df[col].mode()[0]
11         df[col].fillna(mode_value, inplace=True)
12         print(f"Imputed '{col}' with mode: {mode_value}")
13
14 # Impute numerical columns with median
15 numerical_cols_to_impute = ['condition', 'odometer', 'mmr']
16 for col in numerical_cols_to_impute:
17     if df[col].isnull().any():
18         median_value = df[col].median()
19         df[col].fillna(median_value, inplace=True)
20         print(f"Imputed '{col}' with median: {median_value}")
21
22 # Verify no more nulls
23 print("\nVerifying nulls after resolution:")
24 display(df.isnull().sum()[df.isnull().sum() > 0])
```

Dropped 16 rows with nulls in 'vin', 'sellingprice', or 'saledate'.  
 Imputed 'transmission' with mode: automatic  
 /tmp/ipykernel\_8034/917981616.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c  
 The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df[col].fillna(mode_value, inplace=True)
```

Imputed 'body' with mode: Sedan  
 Imputed 'trim' with mode: Base  
 Imputed 'model' with mode: Altima  
 Imputed 'make' with mode: Ford  
 Imputed 'color' with mode: black  
 Imputed 'interior' with mode: black  
 Imputed 'condition' with median: 35.0  
 /tmp/ipykernel\_8034/917981616.py:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c  
 The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df[col].fillna(median_value, inplace=True)
```

Imputed 'odometer' with median: 52255.0  
 Imputed 'mmr' with median: 12250.0

Verifying nulls after resolution:

0

**dtype:** int64

```
1 # Count number of duplicate records
2 duplicate_rows = df.duplicated().sum()
3 print(f"Number of duplicate records found: {duplicate_rows}")
4
5 # Delete duplicate records if any present
6 if duplicate_rows > 0:
7     df.drop_duplicates(inplace=True)
8     print(f"Deleted {duplicate_rows} duplicate records. New dataset shape: {df.shape}")
9 else:
10     print("No duplicate records found.")
```

Number of duplicate records found: 0  
 No duplicate records found.

## 2. Dataframes Queries

### 2.1 Calculate the average, minimum, and maximum car price

```
1 avg_price = df['sellingprice'].mean()
2 min_price = df['sellingprice'].min()
3 max_price = df['sellingprice'].max()
4
5 print(f"Average Car Price: ${avg_price:,.2f}")
6 print(f"Minimum Car Price: ${min_price:,.2f}")
7 print(f"Maximum Car Price: ${max_price:,.2f}")
```

Average Car Price: \$13,611.36  
 Minimum Car Price: \$1.00  
 Maximum Car Price: \$230,000.00

### 2.2 List all unique colors of cars

```
1 print(df['color'].unique())
```

['white' 'gray' 'black' 'red' 'silver' 'blue' 'brown' 'beige' 'purple'  
 'burgundy' '-' 'gold' 'yellow' 'green' 'charcoal' nan 'orange'  
 'off-white' 'turquoise' 'pink' 'lime' '4802' '9410' '1167' '2172' '14872'  
 '12655' '15719' '6388' '16633' '11034' '2711' '6864' '339' '18384' '9887'  
 '9837' '20379' '20627' '721' '6158' '2817' '5705' '18561' '2846' '9562'  
 '5001']

```
1 # List all unique colors of cars
2 unique_colors = df['color'].unique()
3 print("Unique car colors:")
4 for color in unique_colors:
5     print(f"- {color}")
```

```
Unique car colors:  
- white  
- gray  
- black  
- red  
- silver  
- blue  
- brown  
- beige  
- purple  
- burgundy  
- —  
- gold  
- yellow  
- green  
- charcoal  
- orange  
- off-white  
- turquoise  
- pink  
- lime  
- 4802  
- 9410  
- 1167  
- 2172  
- 14872  
- 12655  
- 15719  
- 6388  
- 16633  
- 11034  
- 6864  
- 339  
- 18384  
- 9887  
- 9837  
- 20627  
- 721  
- 6158  
- 5705  
- 18561  
- 2846  
- 5001
```

### 2.3 Find the number of unique car brands and car models

```
1 # Find the number of unique car brands (make)  
2 unique_makes = df['make'].nunique()  
3 print(f"Number of unique car brands: {unique_makes}")  
4  
5 # Find the number of unique car models  
6 unique_models = df['model'].nunique()  
7 print(f"Number of unique car models: {unique_models}")
```

```
Number of unique car brands: 96  
Number of unique car models: 973
```

### 2.4 Find all car information having selling prices greater than \$165000

```
1 # Find all car information having selling prices greater than $165000  
2 high_price_cars = df[df['sellingprice'] > 165000]  
3  
4 print(f"Number of cars with selling price greater than $165,000: {len(high_price_cars)}")  
5 print("Car information for vehicles with selling price > $165,000:")  
6 display(high_price_cars)
```

Number of cars with selling price greater than \$165,000: 7  
 Car information for vehicles with selling price > \$165,000:

|               | year | make          | model      | trim     | body  | transmission | vin               | state | condition | odometer | color | interior |
|---------------|------|---------------|------------|----------|-------|--------------|-------------------|-------|-----------|----------|-------|----------|
| <b>125095</b> | 2012 | Rolls-Royce   | Ghost      | Base     | Sedan | automatic    | sca664s58cux50727 | fl    | 45.0      | 14316.0  | black | beige    |
| <b>344905</b> | 2014 | Ford          | Escape     | Titanium | SUV   | automatic    | 1fmcu9j98eua23833 | mo    | 43.0      | 27802.0  | green | tan      |
| <b>446949</b> | 2015 | Mercedes-Benz | S-Class    | S65 AMG  | Sedan | automatic    | wddug7kb2fa102347 | ca    | 41.0      | 5277.0   | white | white    |
| <b>538347</b> | 2012 | Rolls-Royce   | Ghost      | Base     | sedan | automatic    | sca664s59cux50803 | ca    | 44.0      | 5215.0   | white | tan      |
| <b>545523</b> | 2013 | Rolls-Royce   | Ghost      | Base     | sedan | automatic    | sca664s52dux52152 | fl    | 42.0      | 7852.0   | white | beige    |
| <b>548169</b> | 2011 | Ferrari       | 458 Italia | Base     | coupe | automatic    | zff67nfa1b0178698 | fl    | 46.0      | 12116.0  | red   | black    |
| <b>557570</b> | 2012 | Rolls-Royce   | Ghost      | EWB      | sedan | automatic    | sca664l50cux65625 | ca    | 36.0      | 11832.0  | white | black    |

## 2.5 Find the top 5 most frequently sold car models?

```
1 #Find the top 5 most frequently sold car models?
2 top_5_models = df['model'].value_counts().head(5)
3 print("Top 5 most frequently sold car models:")
4 display(top_5_models)
```

Top 5 most frequently sold car models:

|               | count |
|---------------|-------|
| model         |       |
| <b>Altima</b> | 19349 |
| <b>F-150</b>  | 14479 |
| <b>Fusion</b> | 12946 |
| <b>Camry</b>  | 12545 |
| <b>Escape</b> | 11861 |

dtype: int64

## 2.6 Average selling price by brand (make)

```
1 average_price_by_make = df.groupby('make')['sellingprice'].mean().sort_values(ascending=False)
2 print("Average selling price by brand:")
3 display(average_price_by_make)
```

Average selling price by brand:

| sellingprice       |               |
|--------------------|---------------|
| make               |               |
| <b>Rolls-Royce</b> | 153488.235294 |
| <b>Ferrari</b>     | 127210.526316 |
| <b>Lamborghini</b> | 112625.000000 |
| <b>Bentley</b>     | 74367.672414  |
| <b>airstream</b>   | 71000.000000  |
| ...                | ...           |
| <b>dodge tk</b>    | 550.000000    |
| <b>Geo</b>         | 528.947368    |
| <b>dot</b>         | 500.000000    |
| <b>ford truck</b>  | 483.333333    |
| <b>Daewoo</b>      | 400.000000    |

96 rows × 1 columns

dtype: float64

## 2.7 Minimum selling price of cars for each interior

```
1 min_price_by_interior = df.groupby('interior')['sellingprice'].min().sort_values()
2 print("Minimum selling price by interior:")
3 display(min_price_by_interior)
```

Minimum selling price by interior:

| sellingprice     |        |
|------------------|--------|
| interior         |        |
| <b>black</b>     | 1.0    |
| <b>gray</b>      | 1.0    |
| <b>green</b>     | 100.0  |
| <b>beige</b>     | 100.0  |
| <b>tan</b>       | 100.0  |
| <b>blue</b>      | 150.0  |
| <b>silver</b>    | 150.0  |
| <b>—</b>         | 150.0  |
| <b>burgundy</b>  | 175.0  |
| <b>red</b>       | 200.0  |
| <b>purple</b>    | 200.0  |
| <b>brown</b>     | 200.0  |
| <b>gold</b>      | 250.0  |
| <b>white</b>     | 350.0  |
| <b>off-white</b> | 900.0  |
| <b>orange</b>    | 1200.0 |
| <b>yellow</b>    | 3100.0 |

dtype: float64

## 2.8 Find highest odometer reading per year from highest to lowest order?

```
1 highest_odometer_per_year = df.groupby('year')['odometer'].max().sort_values(ascending=False)
2 print("Highest odometer reading per year (from highest to lowest):")
3 display(highest_odometer_per_year)
```



Highest odometer reading per year (from highest to lowest):

|      | odometer |
|------|----------|
| year |          |
| 1997 | 999999.0 |
| 1996 | 999999.0 |
| 2014 | 999999.0 |
| 2013 | 999999.0 |
| 1998 | 999999.0 |
| 1999 | 999999.0 |
| 1993 | 999999.0 |
| 2010 | 999999.0 |
| 2009 | 999999.0 |
| 2012 | 999999.0 |
| 2008 | 999999.0 |
| 2007 | 999999.0 |
| 2005 | 999999.0 |
| 2006 | 999999.0 |
| 2004 | 999999.0 |
| 2003 | 999999.0 |
| 2002 | 999999.0 |
| 2001 | 999999.0 |
| 2011 | 999999.0 |
| 2000 | 462579.0 |
| 1995 | 396793.0 |
| 1992 | 379307.0 |
| 1994 | 356402.0 |
| 1991 | 274149.0 |
| 1986 | 273327.0 |
| 1990 | 265498.0 |
| 1988 | 241751.0 |
| 1987 | 230946.0 |
| 1989 | 218052.0 |
| 1984 | 170799.0 |
| 1985 | 121644.0 |
| 2015 | 100454.0 |
| 1982 | 85738.0  |
| 1983 | 42697.0  |

dtype: float64

## 2.9 Create a new column for car age (assuming the current year is 2025)

```
1 ##Create a new column for car age (assuming the current year is 2025)
2 df['car_age'] = 2025 - df['year']
3 print("DataFrame with new 'car_age' column:")
4 display(df[['year', 'car_age']].head())
```

DataFrame with new 'car\_age' column:

|   | year | car_age |
|---|------|---------|
| 0 | 2015 | 10      |
| 1 | 2015 | 10      |
| 2 | 2014 | 11      |
| 3 | 2015 | 10      |
| 4 | 2014 | 11      |

## 2.10 Find the number of cars having a condition greater than or equal to 48 and odometer greater than 90000?

```
1 filtered_cars = df[(df['condition'] >= 48) & (df['odometer'] > 90000)]
2
3 print('Count:', len(filtered_cars))
```

Count: 746

```
1 filtered_cars = df[(df['condition'] >= 48) & (df['odometer'] > 90000)]
2 num_filtered_cars = len(filtered_cars)
3 print(f"Number of cars with condition >= 48 and odometer > 90000: {num_filtered_cars}")
```

Number of cars with condition >= 48 and odometer > 90000: 746

## 2.11 Which state consistently has higher car prices for new cars (year>2013)?

```
1 newer_cars = df[df['year'] > 2013]
2
3 state_price = newer_cars.groupby('state')['sellingprice'].mean().sort_values(ascending=False)
4
5 print(state_price)
```

```
state
oh      28020.221053
ab      25204.255319
nj      24237.063973
on      22962.560386
qc      22722.938144
...
3vwd17aj5fm225953  13200.000000
3vwd17aj8fm239622  13200.000000
3vwd17aj3fm276741  13200.000000
3vwd17aj4fm201708  13200.000000
ok       10600.000000
Name: sellingprice, Length: 64, dtype: float64
```

## 2.12 For cars with excellent condition (top 20%), which makes have the lowest average price (value for money)?

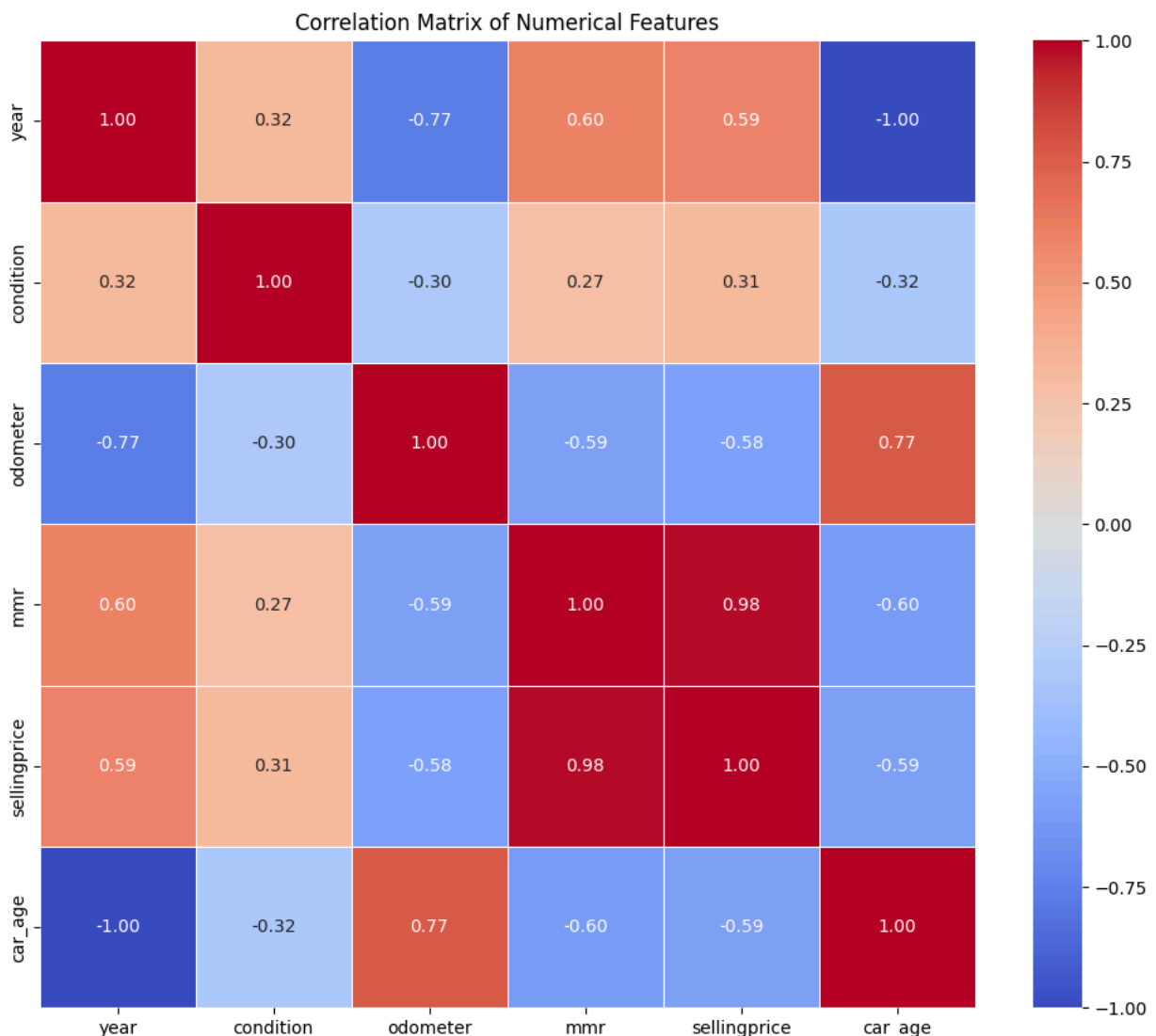
```
1 threshold = df['condition'].quantile(0.80)
2
3 excellent_cars = df[df['condition'] >= threshold]
4
5 best_value = excellent_cars.groupby('make')['sellingprice'].mean().sort_values()
6
7 print(best_value.head(10))
```

```
make
Isuzu      1125.000000
Oldsmobile 1910.000000
honda     4233.333333
Saturn     5700.406504
subaru     6200.000000
chrysler   6225.000000
smart      6835.759494
mazda      7275.000000
Pontiac    7686.824324
Saab       7711.111111
Name: sellingprice, dtype: float64
```

### 3. Data Visualization and insights:

3.1 Show the correlation of all the features(columns) having numerical values(e.g. selling price, odometer).

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Select only numerical columns for correlation analysis
5 numerical_df = df.select_dtypes(include=['number'])
6
7 # Calculate the correlation matrix
8 correlation_matrix = numerical_df.corr()
9
10 # Plotting the heatmap
11 plt.figure(figsize=(12, 10))
12 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
13 plt.title('Correlation Matrix of Numerical Features')
14 plt.show()
```



#### Insight:

- Selling price generally decreases with higher odometer readings.
- Newer cars tend to have higher selling prices.
- Condition score positively impacts selling price.

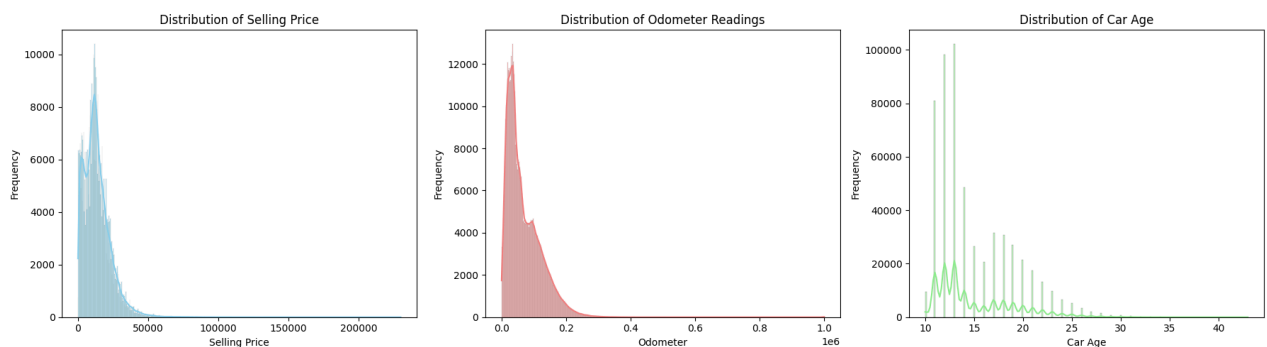
3.1 Visualize the distribution of selling prices, odometer readings, and car age using histograms or density plots.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Set up the matplotlib figure and axes
5 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```

6
7 # Plotting Selling Price Distribution
8 sns.histplot(df['sellingprice'], kde=True, ax=axes[0], color='skyblue')
9 axes[0].set_title('Distribution of Selling Price')
10 axes[0].set_xlabel('Selling Price')
11 axes[0].set_ylabel('Frequency')
12
13 # Plotting Odometer Readings Distribution
14 sns.histplot(df['odometer'], kde=True, ax=axes[1], color='lightcoral')
15 axes[1].set_title('Distribution of Odometer Readings')
16 axes[1].set_xlabel('Odometer')
17 axes[1].set_ylabel('Frequency')
18
19 # Plotting Car Age Distribution
20 sns.histplot(df['car_age'], kde=True, ax=axes[2], color='lightgreen')
21 axes[2].set_title('Distribution of Car Age')
22 axes[2].set_xlabel('Car Age')
23 axes[2].set_ylabel('Frequency')
24
25 plt.tight_layout()
26 plt.show()

```



#### Insight:

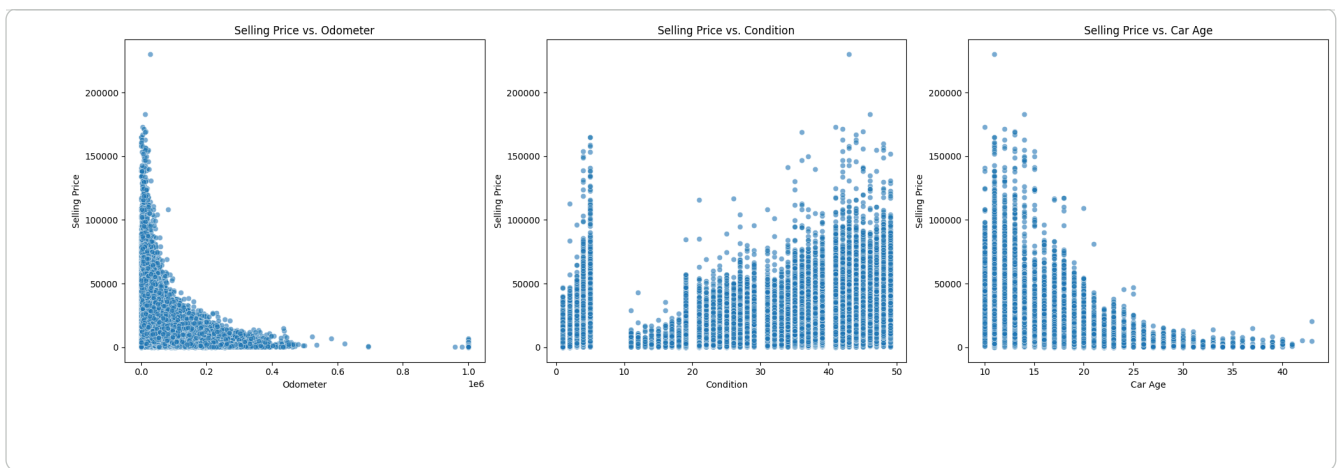
- Newer model cars generally have higher selling prices.
- Older vehicles show significant depreciation.

### 3.1 Analyze the relationship between 'sellingprice' and other key features like 'odometer', 'condition', and 'car\_age' using scatter plots.

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 fig, axes = plt.subplots(1, 3, figsize=(20, 6))
5
6 # Scatter plot: Selling Price vs. Odometer
7 sns.scatterplot(data=df, x='odometer', y='sellingprice', ax=axes[0], alpha=0.6)
8 axes[0].set_title('Selling Price vs. Odometer')
9 axes[0].set_xlabel('Odometer')
10 axes[0].set_ylabel('Selling Price')
11
12 # Scatter plot: Selling Price vs. Condition
13 sns.scatterplot(data=df, x='condition', y='sellingprice', ax=axes[1], alpha=0.6)
14 axes[1].set_title('Selling Price vs. Condition')
15 axes[1].set_xlabel('Condition')
16 axes[1].set_ylabel('Selling Price')
17
18 # Scatter plot: Selling Price vs. Car Age
19 sns.scatterplot(data=df, x='car_age', y='sellingprice', ax=axes[2], alpha=0.6)
20 axes[2].set_title('Selling Price vs. Car Age')
21 axes[2].set_xlabel('Car Age')
22 axes[2].set_ylabel('Selling Price')
23
24 plt.tight_layout()
25 plt.show()

```



### 1. Selling Price vs Odometer

- Cars with lower odometer readings have higher selling prices.
- Selling price decreases as mileage increases.
- High mileage cars usually have lower resale value.

### 2. Selling Price vs Condition

- Cars with better condition scores have higher selling prices.
- Poor condition cars are sold at lower prices.
- Good condition increases resale value.

### 3. Selling Price vs Car Age

- Newer cars have higher selling prices.
- Older cars have lower selling prices.
- Selling price decreases as car age increases due to depreciation.

#### \*\* Overall Observation\*\*

- Low mileage + good condition + newer cars = higher selling price.
- High mileage + old cars + poor condition = lower selling price.

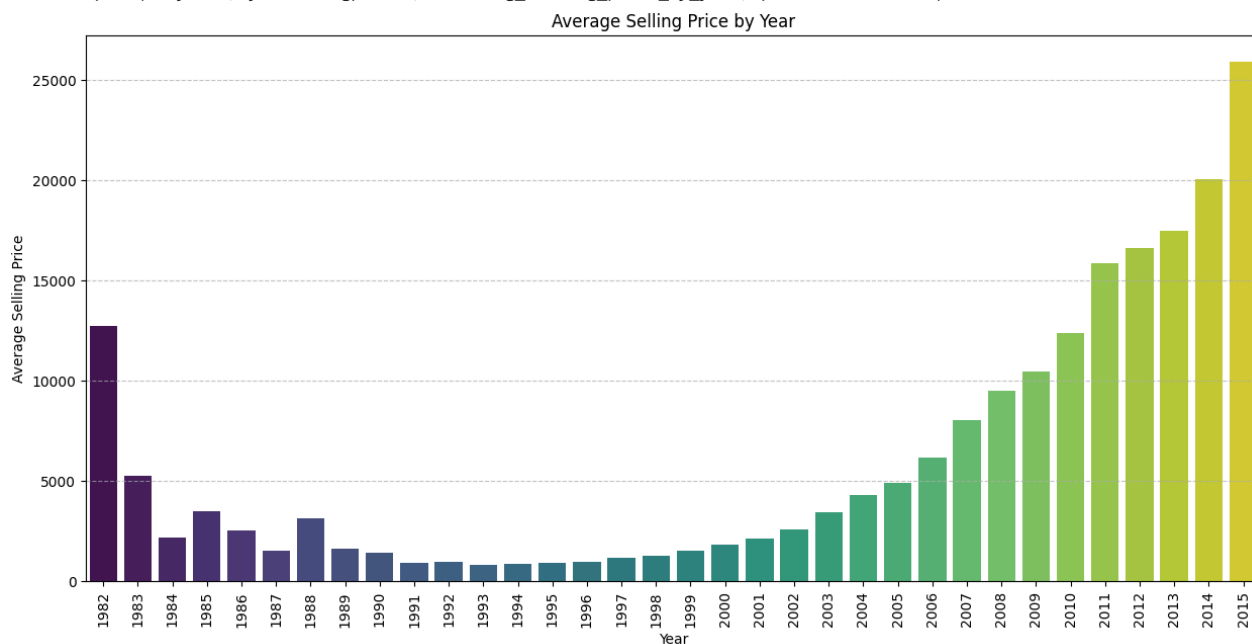
## 3.2 Plot a graph to show the average selling price by year and explain the pattern you notice from the graph. Which plot will you use bar or scatter?

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Calculate the average selling price by year
5 avg_selling_price_by_year = df.groupby('year')['sellingprice'].mean().reset_index()
6
7 # Create the bar plot
8 plt.figure(figsize=(15, 7))
9 sns.barplot(x='year', y='sellingprice', data=avg_selling_price_by_year, palette='viridis')
10 plt.title('Average Selling Price by Year')
11 plt.xlabel('Year')
12 plt.ylabel('Average Selling Price')
13 plt.xticks(rotation=90) # Rotate x-axis labels for better readability
14 plt.grid(axis='y', linestyle='--', alpha=0.7)
15 plt.show()
```

/tmp/ipykernel\_8034/937825218.py:9: FutureWarning:

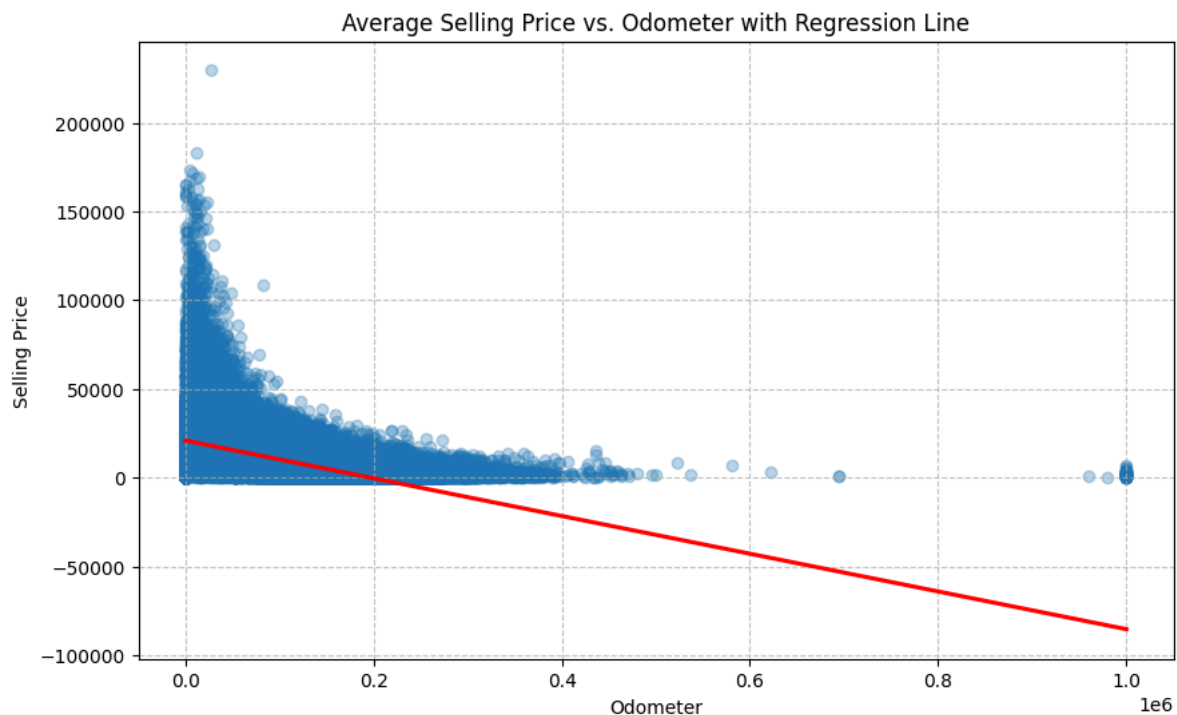
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

`sns.barplot(x='year', y='sellingprice', data=avg_selling_price_by_year, palette='viridis')`



### 3.3 Plot a graph to show average selling price by odometer and explain the trend you notice from the graph.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 plt.figure(figsize=(10, 6))
5 sns.regplot(data=df, x='odometer', y='sellingprice', scatter_kws={'alpha':0.3}, line_kws={'color':'red'})
6 plt.title('Average Selling Price vs. Odometer with Regression Line')
7 plt.xlabel('Odometer')
8 plt.ylabel('Selling Price')
9 plt.grid(True, linestyle='--', alpha=0.7)
10 plt.show()
```

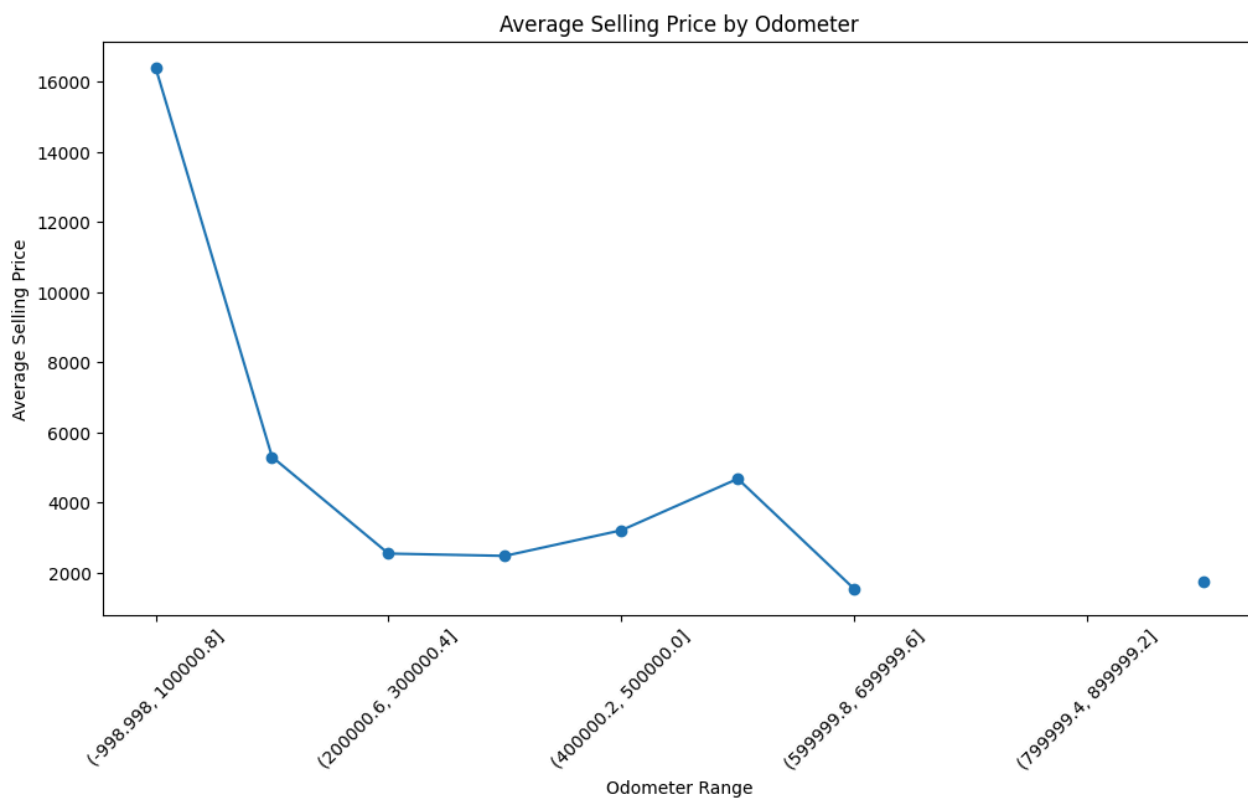


### 3.3 Average Selling Price by Odometer

```

1 odometer_bins = pd.cut(df['odometer'], bins=10)
2
3 avg_price_odometer = df.groupby(odometer_bins, observed=False)['sellingprice'].mean()
4
5 plt.figure(figsize=(12,6))
6 avg_price_odometer.plot(kind='line', marker='o')
7 plt.title('Average Selling Price by Odometer')
8 plt.xlabel('Odometer Range')
9 plt.ylabel('Average Selling Price')
10 plt.xticks(rotation=45)
11 plt.show()

```



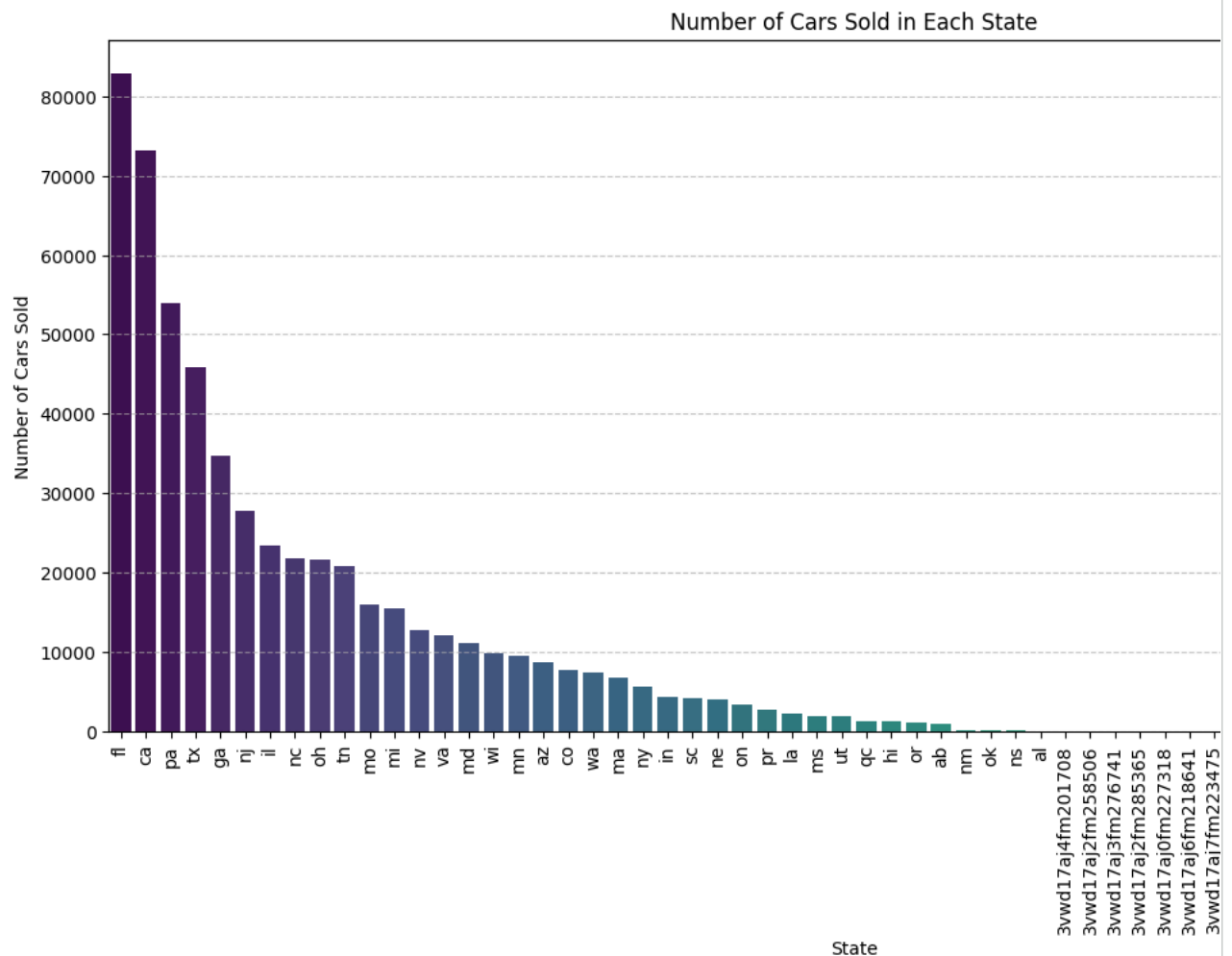
### Simple Insights from the Graph

- Selling price decreases as odometer reading increases.
- Cars with low mileage have higher selling prices.
- High mileage cars are generally sold at lower prices.
- The red regression line shows a negative relationship between mileage and price.
- Most expensive cars are concentrated at lower odometer values. A few outliers are present with unusual prices.

### 3.4 Plot a graph to show the number of cars sold in each state. Find the top three highest car selling states by having a look at the graph?

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Calculate the number of cars sold in each state
5 cars_by_state = df['state'].value_counts().reset_index()
6 cars_by_state.columns = ['state', 'num_cars_sold']
7
8 # Plotting the bar graph
9 plt.figure(figsize=(15, 7))
10 sns.barplot(x='state', y='num_cars_sold', data=cars_by_state, palette='viridis', hue='state', legend=False)
11 plt.title('Number of Cars Sold in Each State')
12 plt.xlabel('State')
13 plt.ylabel('Number of Cars Sold')
14 plt.xticks(rotation=90) # Rotate x-axis labels for better readability
15 plt.grid(axis='y', linestyle='--', alpha=0.7)
16 plt.show()
17
18 # To explicitly find the top 3 states:
19 print('\nTop 3 highest car selling states:')
20 display(cars_by_state.head(3))
```





Top 3 highest car selling states:

|   | state | num_cars_sold |  |
|---|-------|---------------|--|
| 0 | fl    | 82945         |  |
| 1 | ca    | 73148         |  |
| 2 | pa    | 53907         |  |

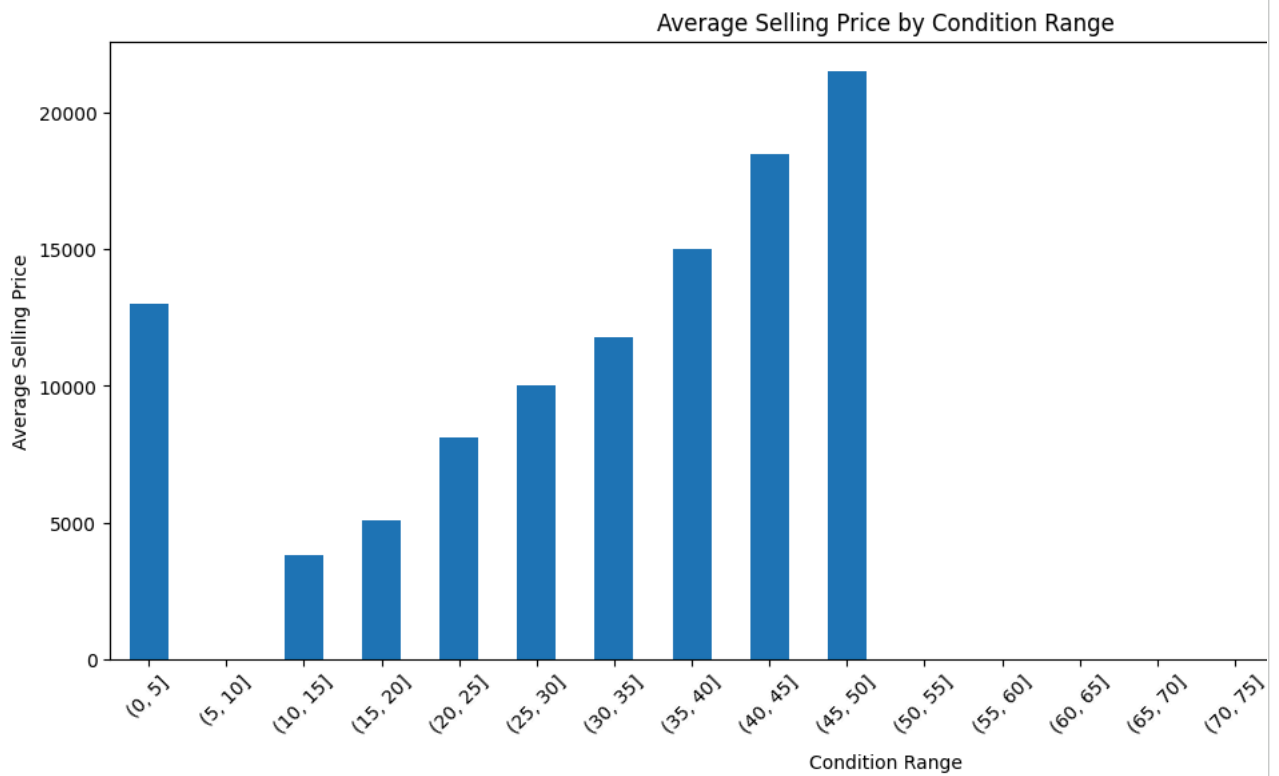
### Insights for Average Selling Price by Condition Range Graph

- Average selling price increases as the condition score increases. Cars in better condition have higher resale value.
- Poor condition cars are sold at lower prices. The highest average selling price is observed in the top condition ranges (40–50).
- Condition plays an important role in determining car price.
- Buyers are willing to pay more for well-maintained vehicles.

```

1 df['condition_range'] = pd.cut(df['condition'], bins=range(0, 101, 5))
2
3 condition_price = df.groupby('condition_range', observed=False)['sellingprice'].mean()
4
5 plt.figure(figsize=(15,6))
6 condition_price.plot(kind='bar')
7 plt.title('Average Selling Price by Condition Range')
8 plt.xlabel('Condition Range')
9 plt.ylabel('Average Selling Price')
10 plt.xticks(rotation=45)
11 plt.show()

```



#### Insights for Number of Cars Sold in Each State Graph

- A few states dominate the used car market.
- Florida (FL), California (CA), and Texas (TX) are the top car selling states.
- Most states have lower sales compared to the top states.
- Car sales are higher in states with larger populations and higher vehicle demand.
- The graph shows uneven distribution of car sales across states.

### 3.5 Plot a bar graph of average selling price by condition score ranges of size

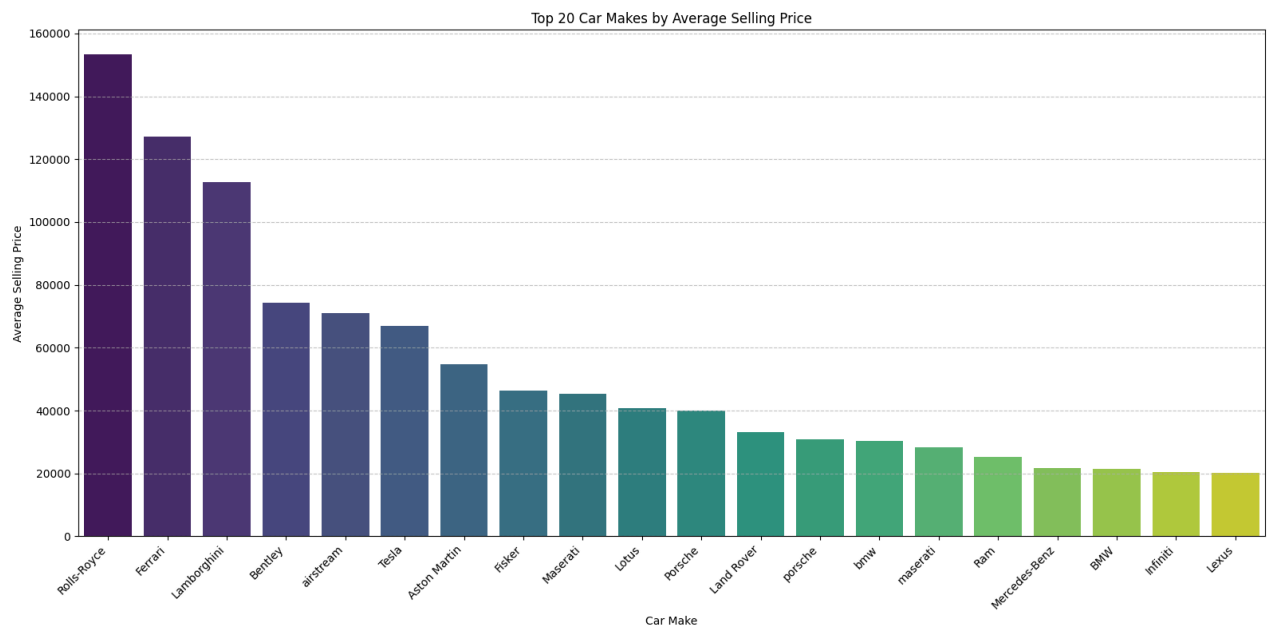
#### 5. Give a summary of the insights.

### 3.7 Plot a graph to show the average selling price by make

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Calculate the average selling price by make
5 avg_selling_price_by_make = df.groupby('make')['sellingprice'].mean().sort_values(ascending=False).reset_index()
6
7 # Plotting the bar graph for the top N makes for better readability
8 plt.figure(figsize=(16, 8))
9 sns.barplot(x='make', y='sellingprice', data=avg_selling_price_by_make.head(20), palette='viridis', hue='make', legend=True)
10 plt.title('Top 20 Car Makes by Average Selling Price')
11 plt.xlabel('Car Make')
12 plt.ylabel('Average Selling Price')
13 plt.xticks(rotation=45, ha='right')
14 plt.grid(axis='y', linestyle='--', alpha=0.7)
15 plt.tight_layout()
16 plt.show()

```



### Insights from 'Average Selling Price by Make' Graph:

This bar graph displays the average selling price for the top 20 car makes. Here are some key observations:

- **Luxury Brands Dominate:** Brands like Rolls-Royce, Ferrari, and Lamborghini command significantly higher average selling prices, indicating their premium market positioning.
- **Price Variation Across Brands:** There's a wide range in average selling prices, reflecting differences in brand perception, vehicle segments, features, and target demographics.
- **Mid-Range Market:** Many popular brands fall into a mid-range average selling price bracket, catering to a broader consumer base.
- **Insights on Value:** Brands with lower average selling prices might represent more accessible or value-oriented options in the market.

### 3.6 Plot a bar graph of no. of cars sold by condition ranges of size 10. Give some insights from the graph.

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Create condition ranges of size 10
5 df['condition_range_10'] = pd.cut(df['condition'], bins=range(0, 101, 10), right=False)
6
7 # Count the number of cars in each condition range
8 cars_by_condition_range = df['condition_range_10'].value_counts().sort_index(ascending=True)
9
10 # Plotting the bar graph
11 plt.figure(figsize=(15, 7))
12 sns.barplot(x=cars_by_condition_range.index.astype(str), y=cars_by_condition_range.values, palette='viridis', hue=cars_t
13 plt.title('Number of Cars Sold by Condition Ranges (Size 10)')
14 plt.xlabel('Condition Range')
15 plt.ylabel('Number of Cars Sold')
16 plt.xticks(rotation=45, ha='right')
17 plt.grid(axis='y', linestyle='--', alpha=0.7)
18 plt.tight_layout()
19 plt.show()

```

